

LABORATORIO 13

Ordenar sin particionar

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo.

Introducción

1 · El tema

Una forma conocida de no leer una tabla entera es particionarla, guardar los datos separados por un valor, por ejemplo el mes. Pero particionar no siempre se puede ni siempre conviene. En este laboratorio vas a ver otra palanca, **ordenar**, y lo que hace falta para que funcione.

La pregunta es si una tabla que no se puede particionar está condenada a leerse entera cada vez.

2 · El problema, en el almacén

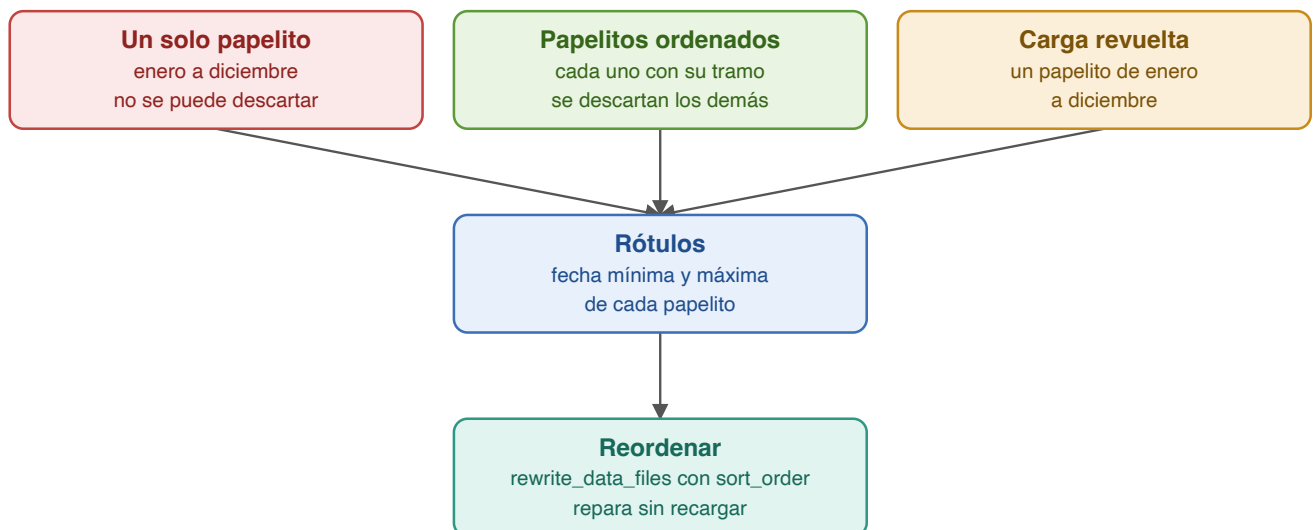
El almacén guarda un año de documentos en un solo pasillo, sin rótulos por mes. Cuando alguien pregunta por junio, el almacenero no tiene cómo saber dónde está junio y revisa caja por caja el año completo.

Poner un pasillo por mes serviría, pero a veces no hay un buen criterio para separar, o el criterio cambia seguido, o separar dejaría cientos de cajitas casi vacías.

3 · El problema en términos técnicos

Iceberg guarda de cada archivo el valor más chico y el más grande de cada columna. Con eso el motor puede descartar un archivo sin abrirlo, si su rango no toca lo que se busca. Pero ese descarte solo funciona si hay varios archivos y cada uno cubre un rango angosto. Una tabla en un solo archivo, o con archivos que mezclan fechas de todo el año, obliga a leerlo todo.

4 · El diagrama



Un solo papelito (rojo). Toda la tabla en un archivo que va del 1 de enero al 31 de diciembre. Su rótulo no deja descartar nada, así que cualquier pregunta lo lee entero.

Papelitos ordenados (verde). La misma tabla partida en varios archivos, ordenada por fecha. Cada archivo cubre un tramo corto del año, y para responder por junio basta abrir los que tocan junio.

Carga revuelta (amarillo). Una carga nueva que llega sin orden deja un archivo que otra vez va de enero a diciembre. Ese archivo entra en todas las preguntas.

Rótulos (azul). El motor decide qué leer mirando la fecha mínima y máxima de cada papelito, que están anotadas en los manifiestos. No abre el archivo para saberlo.

Reordenar (verde agua). `rewrite_data_files` con un orden reescribe lo que ya está, ordenado y en papelitos del tamaño que se le pida. No hay que recargar nada.

5 · La solución

En el almacén, sin crear pasillos nuevos, el almacenero deja las cajas en orden de fecha en el mismo pasillo, en varios montones, y le pone a cada montón un rótulo con la primera y la última fecha que tiene. Cuando alguien pregunta por junio, mira los rótulos por fuera y abre solo los montones que pueden tener junio. Si llega una caja revuelta, la vuelve a ordenar.

En Iceberg eso se hace con `rewrite_data_files` y la estrategia `sort`. Reescribe los archivos ordenados por la columna que le digas y en archivos del tamaño que le pidas. Los rótulos de cada archivo quedan angostos, y el motor descarta los que no sirven sin abrirlos. No se declara ninguna partición.

6 · Los pasos

- **Paso 0.** Creas una tabla ordenada por fecha, pero sin particiones, y ves que queda en un solo archivo.
- **Paso 1.** La partes en varios archivos ordenados con `rewrite_data_files`.
- **Paso 2.** Cuentas cuántos archivos y filas habría que leer para responder por junio.
- **Paso 3.** Comparas lo que leía la tabla de un solo archivo con lo que lee la ordenada.
- **Paso 4.** Llega una carga desordenada y vuelves a medir.
- **Paso 5.** Reordenas lo que ya está escrito y mides por última vez.
- **Cierre.** Las mediciones lado a lado y cuándo conviene ordenar en vez de particionar.

Si quieres repetir el laboratorio desde cero, borra la tabla con el comando de abajo, desde la carpeta del repositorio. El paso 0 igual empieza con un `DROP TABLE IF EXISTS`.

```
bash
bin/reiniciar-lab.sh 13
```

Paso 0 · Una tabla ordenada, sin particiones

Celda 0.1 · Entrar a tu espacio

La pregunta. ¿En qué espacio de trabajo vas a crear la tabla?

Por qué ahora. Las celdas que siguen nombran la tabla sin el espacio adelante, y eso funciona solo si estás parado en el tuyo.

En el almacén. Es pararse frente a tu propio estante antes de empezar.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. Todo lo que va después de dos guiones lo ignora el motor.
- `USE mi_espacio` te deja parado en tu espacio de trabajo, `mi_espacio`.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:10:30 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. La sentencia funcionó. `Listo. La sentencia se ejecutó.` es lo que escribe el cuaderno cuando una sentencia no devuelve filas.

Las tres primeras líneas no son errores. Salen solo en la primera celda que usa Spark, cuando arranca.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:10:30` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.

Celda 0.2 · Partir de cero

La pregunta. ¿Hay una tabla de una corrida anterior que estorbe?

Por qué ahora. Las mediciones solo salen iguales si la tabla parte vacía.

En el almacén. Es sacar del estante la libreta vieja.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` hace que no reclame si la tabla no existe.
- `documentos_lab13` es el nombre de la tabla.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS documentos_lab13
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se borró, o no existía. En los dos casos sale lo mismo.

Celda 0.3 · Crear la tabla ordenada

La pregunta. ¿Qué pasa si creas la tabla con los documentos ordenados por fecha, pero sin particiones?

Por qué ahora. Es el punto de partida del laboratorio. Una tabla ordenada por dentro y sin `PARTITIONED BY`.

En el almacén. Es guardar el año entero en orden de fecha, pero todo en una sola caja.

La sentencia, parte por parte.

- `CREATE TABLE documentos_lab13` crea la tabla.
- `USING iceberg` hace que sea una tabla Iceberg.
- `AS SELECT * FROM curso.dte_2024` la crea con las columnas de esa consulta y la llena con su resultado. `curso.dte_2024` es la tabla de referencia con los documentos de 2024, en el espacio `curso`.
- `ORDER BY fecha_emision` escribe las filas ordenadas por fecha de emisión.
- No lleva `PARTITIONED BY`. No hay cajones.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE documentos_lab13 USING iceberg AS
SELECT * FROM curso.dte_2024
ORDER BY fecha_emision
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla existe y tiene los documentos adentro.

Celda 0.4 · Contar los documentos

La pregunta. ¿Cuántos documentos quedaron?

Por qué ahora. Antes de medir cuánto se lee, hay que saber cuánto hay.

En el almacén. Es contar los renglones de la caja.

La sentencia, parte por parte.

- `count(*)` cuenta las filas.
- `AS documentos` le pone nombre a la columna del resultado.
- `FROM documentos_lab13` es la tabla de hoy.

```
sql
%%sql
-- Celda 0.4
SELECT count(*) AS documentos FROM documentos_lab13
```

Lo que sale en pantalla.

```
text
documentos
30000
1 fila.
```

Cómo se lee. La tabla tiene 30000 documentos, un año completo.

Celda 0.5 · En cuántos archivos quedaron

La pregunta. ¿En cuántos papelitos quedó la tabla, y qué tramo de fechas cubre cada uno?

Por qué ahora. Esta celda decide el laboratorio. Lo que el motor puede descartar depende de cuántos papelitos hay y qué rango cubre cada uno.

En el almacén. Es mirar los rótulos de las cajas, sin abrirlas.

La sentencia, parte por parte.

- `record_count AS filas` es cuántas filas trae cada papelito.
- `readable_metrics.fecha_emision.lower_bound AS desde` es la fecha más chica del papelito. `readable_metrics` es la parte de la vista `.files` con los rótulos de cada columna en forma legible, y `lower_bound` es el mínimo.
- `readable_metrics.fecha_emision.upper_bound AS hasta` es la fecha más grande. `upper_bound` es el máximo.
- `FROM mi_espacio.documentos_lab13.files` es la vista de sistema con los papelitos vigentes, nombrada con tres partes, el espacio, la tabla y la vista.
- `ORDER BY desde` ordena los papelitos por su fecha más chica.

```
sql
%%sql
-- Celda 0.5
SELECT record_count AS filas,
       readable_metrics.fecha_emision.lower_bound AS desde,
       readable_metrics.fecha_emision.upper_bound AS hasta
FROM mi_espacio.documentos_lab13.files
ORDER BY desde
```

Lo que sale en pantalla.

```
text
filas desde hasta
30000 2024-01-01 2024-12-31
1 fila.
```

Cómo se lee. Un solo papelito con las 30000 filas, que va del 2024-01-01 al 2024-12-31.

La tabla está ordenada por dentro, pero eso no le sirve a nadie. El motor decide qué leer mirando papelitos enteros, y hay uno solo. Para responder por junio tiene que abrirlo, y con él entra el año completo.

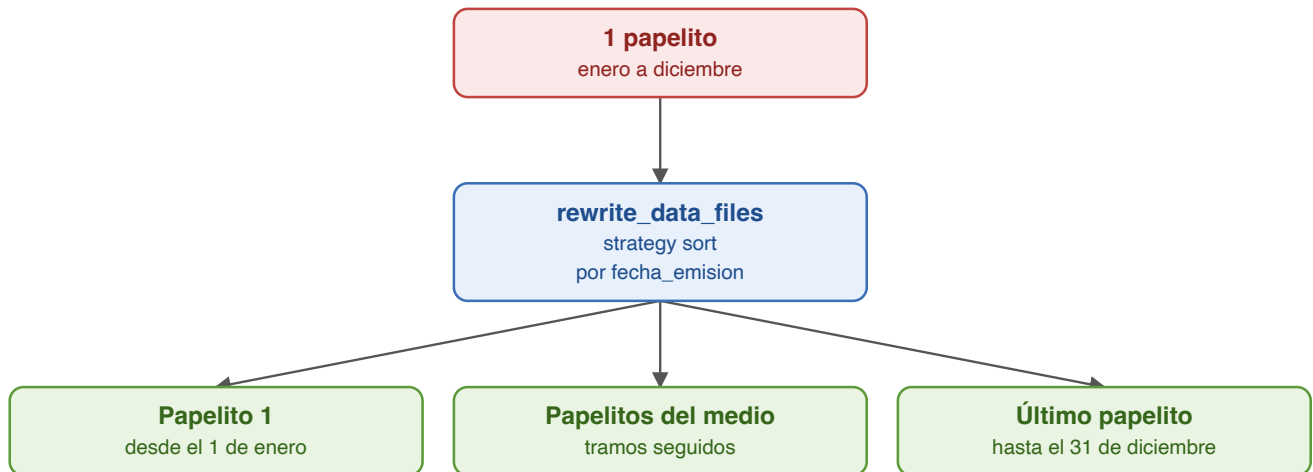
Paso 1 · Partirla en varios archivos

Celda 1.1 · Reescribir ordenado

La pregunta. ¿Cómo se parte la tabla en varios papelitos, cada uno con un tramo de fechas?

Por qué ahora. Ordenar sin tener varios archivos no hace nada. Hay que partirla.

En el almacén. Es vaciar la caja grande y repartir los documentos en cajas más chicas, en orden de fecha, y ponerle a cada una su rótulo.



1 papelito (rojo). La tabla como quedó en el paso 0.

rewrite_data_files (azul). El procedimiento lee los papelitos, los ordena por `fecha_emision` y los vuelve a escribir en papelitos de un tamaño objetivo.

Papelito 1, Papelitos del medio y Último papelito (verde). Cada papelito nuevo cubre un tramo corto y seguido del año, sin pisarse con los otros salvo en el día del borde.

La sentencia, parte por parte.

- `CALL spark_catalog.system.rewrite_data_files(...)` llama al procedimiento de Iceberg que reescribe los papelitos de una tabla. `spark_catalog` es el catálogo y `system` el grupo de procedimientos de Iceberg.
- `table => 'mi_espacio.documentos_lab13'` es la tabla, con su espacio, entre comillas. La flecha `=>` pone nombre a cada argumento.
- `strategy => 'sort'` pide reescribir ordenando.
- `sort_order => 'fecha_emision ASC NULLS LAST'` es el orden, por fecha de emisión de menor a mayor, con las fechas vacías al final. `ASC` es ascendente.
- `options => map(...)` son opciones en pares nombre y valor. `map` arma esos pares.
- `'target-file-size-bytes', '250000'` pide papelitos de unos `250000` bytes, que son 0,25 MB. Es chico a propósito, para que salgan varios.
- `'min-input-files', '1'` permite reescribir aunque haya un solo papelito. Por defecto el procedimiento no toca tablas con tan pocos archivos.
- `'rewrite-all', 'true'` pide reescribir todos los papelitos, aunque ya tengan buen tamaño.

```

sql
%%sql
-- Celda 1.1
CALL spark_catalog.system.rewrite_data_files(
  table      => 'mi_espacio.documentos_lab13',
  strategy   => 'sort',
  sort_order => 'fecha_emision ASC NULLS LAST',
  options    => map('target-file-size-bytes', '250000',
                  'min-input-files',         '1',
                  'rewrite-all',            'true')
)

```

Lo que sale en pantalla.

```

text
rewritten_data_files_count  added_data_files_count  rewritten_bytes_count
1                          9                        720233
1 fila.

```

Cómo se lee. Leyó 1 papelito y escribió 9. Movié 720233 bytes, unos 0,72 MB.

- `rewritten_data_files_count` es cuántos papelitos leyó y dejó de usar.
- `added_data_files_count` es cuántos papelitos nuevos escribió.
- `rewritten_bytes_count` es cuántos bytes reescribió.

Celda 1.2 · Los tramos

La pregunta. ¿Qué tramo de fechas cubre ahora cada papelito?

Por qué ahora. Hay que comprobar que la reescritura dejó rangos angostos.

En el almacén. Es leer los rótulos de las cajas nuevas.

La sentencia, parte por parte. Es la misma consulta de la celda 0.5.

- `record_count AS filas` es cuántas filas trae cada papelito.
- `readable_metrics.fecha_emision.lower_bound AS desde` y `readable_metrics.fecha_emision.upper_bound AS hasta` son su fecha mínima y máxima.
- `FROM mi_espacio.documentos_lab13.files` es la vista de papelitos vigentes.
- `ORDER BY desde` ordena por la fecha más chica.

```

sql
%%sql
-- Celda 1.2
SELECT record_count AS filas,
       readable_metrics.fecha_emision.lower_bound AS desde,
       readable_metrics.fecha_emision.upper_bound AS hasta
FROM mi_espacio.documentos_lab13.files
ORDER BY desde

```

Lo que sale en pantalla.

```
text
filas desde hasta
4000 2024-01-01 2024-02-18
4000 2024-02-18 2024-04-06
1791 2024-04-06 2024-04-27
4000 2024-04-28 2024-06-16
4000 2024-06-16 2024-08-04
1807 2024-08-04 2024-08-26
4000 2024-08-27 2024-10-14
4000 2024-10-14 2024-12-02
2402 2024-12-02 2024-12-31
9 filas.
```

Cómo se lee. Nueve papelitos, cada uno con su tramo del año, del más antiguo al más nuevo.

La mayoría tiene 4000 filas. Tres tienen menos, 1791, 1807 y 2402, porque la reescritura se reparte en varias tareas y el último papelito de cada tarea queda con lo que sobró. Los tramos no se pisan, salvo en el día del borde. Por ejemplo, el primero termina el 2024-02-18 y el segundo empieza ese mismo día, porque los documentos de ese día quedaron repartidos entre los dos.

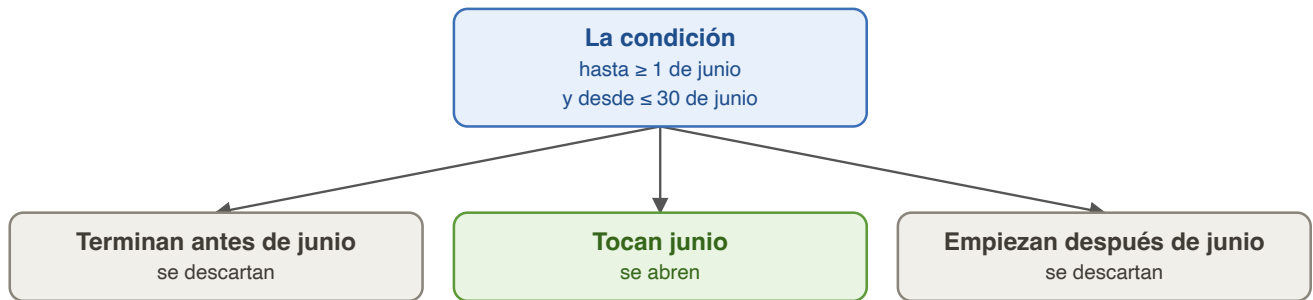
Paso 2 · La regla de medir

Celda 2.1 · Cuánto habría que leer para junio

La pregunta. ¿Cuántos papelitos tendría que abrir el motor para responder por junio?

Por qué ahora. Es la medición que decide si ordenar sirvió.

En el almacén. Es contar las cajas cuyo rótulo toca junio. Las que terminan antes de junio o empiezan después no se abren.



Terminan antes de junio y Empiezan después de junio (gris). Sus rótulos no tocan junio. El motor los descarta sin abrirlos.

Tocan junio (verde). Su tramo se cruza con junio. Hay que abrirlos.

La condición (azul). Un papelito toca junio si termina el 1 de junio o después y empieza el 30 de junio o antes. Es la misma regla que usa el motor.

La sentencia, parte por parte.

- `count(*) AS archivos_a_leer` cuenta los papelitos que pasan el filtro.
- `sum(record_count) AS filas_a_leer` suma las filas que traen.
- `FROM mi_espacio.documentos_lab13.files` es la vista de papelitos vigentes.
- `WHERE readable_metrics.fecha_emision.upper_bound >= DATE '2024-06-01'` se queda con los papelitos que terminan el 1 de junio o después.
- `AND readable_metrics.fecha_emision.lower_bound <= DATE '2024-06-30'` y que empiezan el 30 de junio o antes. `DATE '...'` convierte el texto en fecha.

```
sql
%%sql
-- Celda 2.1
SELECT count(*)          AS archivos_a_leer,
       sum(record_count) AS filas_a_leer
FROM mi_espacio.documentos_lab13.files
WHERE readable_metrics.fecha_emision.upper_bound >= DATE '2024-06-01'
      AND readable_metrics.fecha_emision.lower_bound <= DATE '2024-06-30'
```

Lo que sale en pantalla.

```
text
archivos_a_leer  filas_a_leer
2                8000
1 fila.
```

Cómo se lee. Para responder por junio habría que abrir 2 papelitos y leer 8000 filas.

Son los dos papelitos cuyo tramo cruza junio en la celda 1.2, el que va del 2024-04-28 al 2024-06-16 y el que va del 2024-06-16 al 2024-08-04. Cada uno trae 4000 filas.

Paso 3 · El cuadro completo

Este paso no tiene celdas. En el cuaderno es un cuadro para comparar.

La pregunta. ¿Cuánto ganó la tabla al ordenarla y partirla?

En el almacén. Antes había que abrir la caja del año entero. Ahora basta con abrir dos cajas.

Tabla	Archivos a leer	Filas a leer
Un solo archivo, celda 0.5	1	30000
Ordenada en nueve archivos, celda 2.1	2	8000

Pasó de leer el año entero a leer unos dos meses y medio, sin declarar ninguna partición. Una tabla particionada por mes leería todavía menos, solo junio, porque un cajón por mes es más angosto que un tramo de fechas. Ordenar sirve cuando particionar no es una opción, porque no hay una columna con pocos valores distintos, porque el criterio de consulta cambia seguido, o porque particionar dejaría demasiados archivos chicos.

Paso 4 · Lo que pasa cuando llega una carga desordenada

Celda 4.1 · Una carga revuelta

La pregunta. ¿Qué le pasa al orden cuando llega una carga que no viene ordenada?

Por qué ahora. El orden es frágil, y hay que saberlo antes de apoyarse en él.

En el almacén. Llega una caja con documentos de todo el año mezclados, y el almacenero la pone tal cual en el pasillo.

La sentencia, parte por parte.

- `INSERT INTO documentos_lab13` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae otra vez los documentos del año.
- `ORDER BY rand()` los ordena al azar. `rand()` da un número aleatorio para cada fila, así que el orden cambia en cada corrida.

Es el mismo año otra vez, así que la tabla queda con cada documento repetido. Es a propósito y no importa para lo que se mide, que es cómo quedan repartidos los papelitos.

```
sql
%%sql
-- Celda 4.1
INSERT INTO documentos_lab13
SELECT * FROM curso.dte_2024 ORDER BY rand()
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La carga entró.

Celda 4.2 · Los papelitos después de la carga

La pregunta. ¿Cómo quedaron los tramos después de la carga revuelta?

Por qué ahora. Para ver el daño.

En el almacén. Es leer los rótulos otra vez, con la caja nueva incluida.

La sentencia, parte por parte. Es la misma de la celda 1.2.

- `record_count AS filas` es cuántas filas trae cada papelito.
- `readable_metrics.fecha_emision.lower_bound AS desde` y `readable_metrics.fecha_emision.upper_bound AS hasta` son su fecha mínima y máxima.
- `FROM mi_espacio.documentos_lab13.files` es la vista de papelitos vigentes.
- `ORDER BY desde` ordena por la fecha más chica.

```

sql
%%sql
-- Celda 4.2
SELECT record_count AS filas,
       readable_metrics.fecha_emision.lower_bound AS desde,
       readable_metrics.fecha_emision.upper_bound AS hasta
FROM mi_espacio.documentos_lab13.files
ORDER BY desde

```

Lo que sale en pantalla.

```

text
filas desde hasta
30000 2024-01-01 2024-12-31
4000 2024-01-01 2024-02-18
4000 2024-02-18 2024-04-06
1791 2024-04-06 2024-04-27
4000 2024-04-28 2024-06-16
4000 2024-06-16 2024-08-04
1807 2024-08-04 2024-08-26
4000 2024-08-27 2024-10-14
4000 2024-10-14 2024-12-02
2402 2024-12-02 2024-12-31
10 filas.

```

Cómo se lee. Los nueve papelitos ordenados siguen ahí, y arriba apareció uno nuevo con 30000 filas que va del 2024-01-01 al 2024-12-31.

Sus filas llegaron mezcladas, así que su rótulo cubre el año completo. Ese papelito va a entrar en todas las consultas, pregunten por el mes que pregunten.

Celda 4.3 · Volver a medir junio

La pregunta. ¿Cuánto hay que leer ahora para responder por junio?

Por qué ahora. Para poner un número al daño.

En el almacén. Es contar otra vez las cajas cuyo rótulo toca junio.

La sentencia, parte por parte. Es la misma de la celda 2.1.

- `count(*)` AS `archivos_a_leer` y `sum(record_count)` AS `filas_a_leer` cuentan papelitos y filas.
- `FROM mi_espacio.documentos_lab13.files` es la vista de papelitos vigentes.
- `WHERE ... upper_bound >= DATE '2024-06-01' AND ... lower_bound <= DATE '2024-06-30'` se queda con los que tocan junio.

```

sql
%%sql
-- Celda 4.3
SELECT count(*) AS archivos_a_leer,
       sum(record_count) AS filas_a_leer
FROM mi_espacio.documentos_lab13.files
WHERE readable_metrics.fecha_emision.upper_bound >= DATE '2024-06-01'
      AND readable_metrics.fecha_emision.lower_bound <= DATE '2024-06-30'

```

Lo que sale en pantalla.

```
text
archivos_a_leer  filas_a_leer
3                38000
1 fila.
```

Cómo se lee. Ahora hay que abrir 3 papelitos y leer 38000 filas.

Son los dos de antes, con sus 8000 filas, más el papelito revuelto con sus 30000. Una sola carga desordenada multiplicó lo que se lee.

Paso 5 · Reordenar lo que ya está

Celda 5.1 · Reescribir ordenado otra vez

La pregunta. ¿Se puede arreglar sin recargar ni recrear la tabla?

Por qué ahora. El orden se degrada solo, así que hace falta una forma de repararlo.

En el almacén. Es vaciar todas las cajas, las ordenadas y la revuelta, y volver a repartir en orden.

La sentencia, parte por parte. Es la misma de la celda 1.1.

- `CALL spark_catalog.system.rewrite_data_files(...)` reescribe los papelitos de la tabla.
- `table => 'mi_espacio.documentos_lab13'` es la tabla.
- `strategy => 'sort'` y `sort_order => 'fecha_emision ASC NULLS LAST'` piden ordenar por fecha.
- `'target-file-size-bytes', '250000'` pide papelitos de unos 0,25 MB.
- `'min-input-files', '1'` y `'rewrite-all', 'true'` hacen que reescriba todo.

```
sql
%%sql
-- Celda 5.1
CALL spark_catalog.system.rewrite_data_files(
  table      => 'mi_espacio.documentos_lab13',
  strategy   => 'sort',
  sort_order => 'fecha_emision ASC NULLS LAST',
  options    => map('target-file-size-bytes', '250000',
                  'min-input-files',         '1',
                  'rewrite-all',            'true')
)
```

Lo que sale en pantalla.

```
text
rewritten_data_files_count  added_data_files_count  rewritten_bytes_count
10                          18                          1607543
1 fila.
```

Cómo se lee. Leyó 10 papelitos y escribió 18. Movié 1607543 bytes, unos 1,61 MB.

Son 18 porque ahora hay el doble de documentos, cada uno repetido dos veces por la carga del paso 4.

Celda 5.2 · Los tramos, por tercera vez

La pregunta. ¿Volvieron los tramos angostos?

Por qué ahora. Para comprobar que la reparación funcionó.

En el almacén. Es leer los rótulos de las cajas nuevas.

La sentencia, parte por parte. Es la misma de la celda 1.2.

- `record_count AS filas`, `readable_metrics.fecha_emision.lower_bound AS desde` y `readable_metrics.fecha_emision.upper_bound AS hasta` son las filas y el tramo de cada papelito.
- `FROM mi_espacio.documentos_lab13.files` es la vista de papelitos vigentes.
- `ORDER BY desde` ordena por la fecha más chica.

```
sql
%%sql
-- Celda 5.2
SELECT record_count AS filas,
       readable_metrics.fecha_emision.lower_bound AS desde,
       readable_metrics.fecha_emision.upper_bound AS hasta
FROM mi_espacio.documentos_lab13.files
ORDER BY desde
```

Lo que sale en pantalla.

```
text
filas desde hasta
4000 2024-01-01 2024-01-25
4000 2024-01-25 2024-02-18
2000 2024-02-18 2024-02-29
4000 2024-03-01 2024-03-25
4000 2024-03-25 2024-04-18
1582 2024-04-18 2024-04-27
4000 2024-04-28 2024-05-23
4000 2024-05-23 2024-06-16
2110 2024-06-16 2024-06-28
4000 2024-06-29 2024-07-23
4000 2024-07-23 2024-08-17
1956 2024-08-17 2024-08-29
4000 2024-08-30 2024-09-23
4000 2024-09-23 2024-10-17
2068 2024-10-17 2024-10-29
4000 2024-10-30 2024-11-23
4000 2024-11-23 2024-12-18
2284 2024-12-18 2024-12-31
18 filas.
```

Cómo se lee. Dieciocho papelitos, otra vez en tramos ordenados, ahora más angostos porque hay más documentos por día.

Los cortes exactos de esta celda y las filas de los papelitos más chicos cambian en cada corrida. La carga del paso 4 llega en orden aleatorio, y eso cambia cómo se reparte el trabajo al reescribir. Lo que no cambia es que cada papelito vuelve a cubrir un tramo corto.

Celda 5.3 · Medir junio por última vez

La pregunta. ¿Cuánto hay que leer ahora para responder por junio?

Por qué ahora. Para cerrar la comparación con el paso 4.

En el almacén. Contar las cajas cuyo rótulo toca junio.

La sentencia, parte por parte. Es la misma de la celda 2.1.

- `count(*) AS archivos_a_leer` y `sum(record_count) AS filas_a_leer` cuentan papelitos y filas.
- `FROM mi_espacio.documentos_lab13.files` es la vista de papelitos vigentes.
- `WHERE ... upper_bound >= DATE '2024-06-01' AND ... lower_bound <= DATE '2024-06-30'` se queda con los que tocan junio.

```

sql
%%sql
-- Celda 5.3
SELECT count(*)          AS archivos_a_leer,
       sum(record_count) AS filas_a_leer
FROM mi_espacio.documentos_lab13.files
WHERE readable_metrics.fecha_emision.upper_bound >= DATE '2024-06-01'
      AND readable_metrics.fecha_emision.lower_bound <= DATE '2024-06-30'

```

Lo que sale en pantalla.

```

text
archivos_a_leer  filas_a_leer
3                10110
1 fila.

```

Cómo se lee. Hay que abrir 3 papelitos y leer 10110 filas, en vez de las 38000 del paso 4.

Son los tres papelitos de la celda 5.2 que cruzan junio, el que va del 2024-05-23 al 2024-06-16, el del 2024-06-16 al 2024-06-28 y el del 2024-06-29 al 2024-07-23. Este número también cambia un poco en cada corrida, por la carga aleatoria.

Celda 5.4 · Lo que dejó escrito la reorganización

La pregunta. ¿Qué páginas dejaron en la libreta la carga y las dos reescrituras?

Por qué ahora. Para ver que reordenar también queda anotado.

En el almacén. Es mirar la lista de páginas de la libreta.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide el número de cada página, la hora en que se colgó, en UTC, y qué se hizo en ella.
- `FROM mi_espacio.documentos_lab13.snapshots` es la vista de sistema con las páginas.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```

sql
%%sql
-- Celda 5.4
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.documentos_lab13.snapshots
ORDER BY committed_at

```

Lo que sale en pantalla.

```

text
snapshot_id      committed_at      operation
530236845447903924 2026-09-25 18:10:38.144 append
4212473458663648284 2026-09-25 18:10:40.540 replace
8305626539744031014 2026-09-25 18:10:42.002 append
1566782443588773562 2026-09-25 18:10:43.551 replace
4 filas.

```

Cómo se lee. Cuatro páginas, dos `append` y dos `replace`.

- `530236845447903924`, a las 18:10:38.144 UTC, es la creación de la tabla con el año ordenado. `append` significa que se agregaron filas.
- `4212473458663648284`, a las 18:10:40.540 UTC, es la primera reescritura del paso 1. `replace` significa que se cambiaron papelitos por otros con el mismo contenido.
- `8305626539744031014`, a las 18:10:42.002 UTC, es la carga revuelta del paso 4.
- `1566782443588773562`, a las 18:10:43.551 UTC, las 15:10 en Chile, es la reescritura del paso 5.

Reescribir no cambia los datos, pero deja su página, como todo lo que toca la tabla.

Preguntas frecuentes

Si la tabla se creó con ORDER BY, ¿por qué quedó en un solo archivo?

Porque treinta mil documentos pesan menos que el tamaño que Iceberg busca por defecto para un archivo, que es de cientos de MB. Ordenar decide en qué orden quedan las filas dentro del archivo, no en cuántos archivos se reparten. Por eso la celda 1.1 pide un tamaño objetivo chico.

¿Por qué los tramos se tocan en un día?

Porque la reescritura corta cuando el papelito llega al tamaño pedido, no cuando cambia el día. Si el corte cae a mitad de un día, los documentos de ese día quedan en dos papelitos, y los dos rótulos incluyen esa fecha. No es un problema. A lo más obliga a abrir un papelito de más en el borde.

¿Por qué mis números de los pasos 4 y 5 no son iguales a los de esta guía?

Porque la carga del paso 4 usa `ORDER BY rand()`, que ordena al azar y cambia en cada corrida. Eso cambia cómo se reparte la reescritura del paso 5. Los cortes de la celda 5.2 y las filas de la celda 5.3 van a ser parecidos pero no idénticos. Lo que sí se repite es la forma, un papelito revuelto que se mete en todo y una reescritura que lo arregla.

¿Se puede ordenar y particionar a la vez?

Sí, y se combinan bien. Una tabla puede estar particionada por mes y ordenada por RUT dentro de cada partición. El motor descarta primero por mes y después, dentro del mes, por el rango de RUT de cada papelito, y lee todavía menos.

¿Cada cuánto hay que reordenar?

Depende de cuánto llega desordenado. Cada carga sin orden agrega papelitos con rótulos anchos. Se reordena con `rewrite_data_files`, como una rutina de mantención, cuando la medición de la celda 2.1 empieza a subir. Es la misma herramienta que junta papelitos chicos, con un orden adentro.

¿Reescribir cambia los datos?

No. Cambia en qué papelitos están las filas y en qué orden, pero no el contenido. Por eso la página que deja es `replace` y no `overwrite`.

¿Por qué las horas no calzan con mi reloj?

Porque Spark muestra las horas en UTC, la hora universal. Chile en septiembre está tres horas atrás, así que las 18:10 UTC son las 15:10 en Chile.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Una segunda palanca para no leer la tabla entera. Ordenar agrupa por rango, sin declarar ninguna partición, y hace que el motor descarte papelitos mirando solo sus rótulos.

En el almacén. Las cajas en orden de fecha y con su rótulo bastan para no abrir el año entero. Una caja revuelta arruina el orden, y se arregla volviendo a ordenar lo que ya está, sin traer nada de nuevo.



Un archivo (rojo). Con un solo papelito da igual lo ordenado que esté por dentro. Se lee entero.

Ordenada en tramos (verde). Partida en varios papelitos con rangos angostos, el motor descarta los que no tocan lo que se busca.

Carga revuelta (amarillo). El orden se degrada solo. Cada carga desordenada agrega un papelito que abarca todo.

Reordenada (verde agua). `rewrite_data_files` con `sort_order` repara lo escrito sin recargar.

Los números de la solución ejecutada, para responder por junio.

Momento	Archivos a leer	Filas a leer
Un solo archivo, celda 0.5	1	30000
Ordenada en nueve archivos, celda 2.1	2	8000
Después de la carga revuelta, celda 4.3	3	38000
Reordenada, con el doble de documentos, celda 5.3	3	10110

Particionar agrupa por valor exacto y conviene cuando hay una columna con pocos valores distintos y estable. Ordenar agrupa por rango y conviene cuando no hay buen criterio de partición o cambia seguido. Las dos hacen lo mismo en el fondo, que el motor pueda descartar papelitos sin abrirlos.